

COMPTE RENDU

Algorithmique Avancée - TP8 - Parcours de Graphes BFS vs DFS
3e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
Filière : Cybersécurité
Année : 2025/2026
Enseignant : M. BAKHOUYA
Date : 22 novembre 2025

Table des matières

1	Rappel des objectifs du TP	2
2	Analyse théorique des algorithmes	2
2.1	BFS	2
2.1.1	Principe	2
2.1.2	Complexité	2
2.2	DFS	2
2.2.1	Principe	2
2.2.2	Complexité	3
2.3	Tableau comparatif	3
2.4	Note sur la représentation	3
3	Résultats expérimentaux	4
3.1	Impact de la densité	4
3.2	Graphes peu denses (Sparse - 10%)	5
3.3	Graphes moyennement denses (Medium - 30%)	6
3.4	Graphes très denses (Dense - 70%)	7
3.5	Vue d'ensemble globale	8
3.6	Analyse en échelle log-log	9
4	Analyse quantitative détaillée	10
4.1	Vérification de la complexité $O(V + E)$	10
4.1.1	Croissance en fonction de la taille	10
4.1.2	Croissance en fonction de la densité	10
4.2	Ratios de performance BFS vs DFS	10
5	Conclusion	11

1 Rappel des objectifs du TP

Ce TP8 a pour objectif d'étudier et de comparer expérimentalement **deux algorithmes de parcours de graphes** :

- **BFS** : Parcours en largeur, explore niveau par niveau
- **DFS** : Parcours en profondeur, explore au maximum avant de backtracker

L'objectif principal est de **mesurer et comparer les performances** de ces deux algorithmes en fonction de :

- La **taille du graphe** (nombre de sommets)
- La **densité du graphe** (proportion d'arêtes)

2 Analyse théorique des algorithmes

2.1 BFS

2.1.1 Principe

Le parcours en largeur (BFS) explore le graphe niveau par niveau à partir d'un sommet source :

Fonctionnement :

1. Marquer le sommet de départ comme visité et l'enfiler dans une FIFO (queue)
2. Tant que la queue n'est pas vide :
 - Défiler un sommet
 - Pour chaque voisin non visité : le marquer comme visité et l'enfiler

2.1.2 Complexité

Opération	Complexité	Description
Parcours complet	$O(V + E)$	Visite chaque sommet une fois (V) et explore chaque arête une fois (E)

TABLE 1 – Complexité de BFS

Caractéristiques :

- Trouve le **plus court chemin** (en nombre d'arêtes) depuis la source
- Utilise une **FIFO** (queue) pour gérer l'ordre de visite
- Parcours **niveau par niveau**

2.2 DFS

2.2.1 Principe

Le parcours en profondeur (DFS) explore le graphe en allant aussi loin que possible avant de revenir en arrière :

Fonctionnement (version récursive) :

1. Marquer le sommet actuel comme visité
2. Enregistrer le temps de découverte
3. Pour chaque voisin non visité : appel récursif
4. Enregistrer le temps de fin

2.2.2 Complexité

Opération	Complexité	Description
Parcours complet	$O(V + E)$	Visite chaque sommet une fois (V) et explore chaque arête une fois (E)

TABLE 2 – Complexité de DFS

Caractéristiques :

- Explore en **profondeur** avant d'explorer en largeur
- Utilise la **récursion** (stack implicite) ou une stack explicite
- Utile pour détecter les cycles, composantes connexes, etc.

2.3 Tableau comparatif

Algorithme	Complexité	Structure	Usage typique
BFS	$O(V + E)$	FIFO (queue)	Plus court chemin
DFS	$O(V + E)$	Récursion (stack)	Détection de cycles

TABLE 3 – Comparaison théorique BFS vs DFS

2.4 Note sur la représentation

Ce TP utilise une **représentation par liste d'adjacence**, adaptée aux graphes de densité variable. Une représentation par matrice d'adjacence serait également possible :

- **Matrice d'adjacence** : $O(V^2)$ en espace, $O(1)$ pour tester une arête, idéale pour graphes denses ($> 50\%$)
- **Liste d'adjacence** : $O(V + E)$ en espace, $O(\text{degré})$ pour tester une arête, idéale pour graphes peu denses

Les complexités de BFS et DFS restent $O(V + E)$ quelle que soit la représentation choisie.

3 Résultats expérimentaux

3.1 Impact de la densité

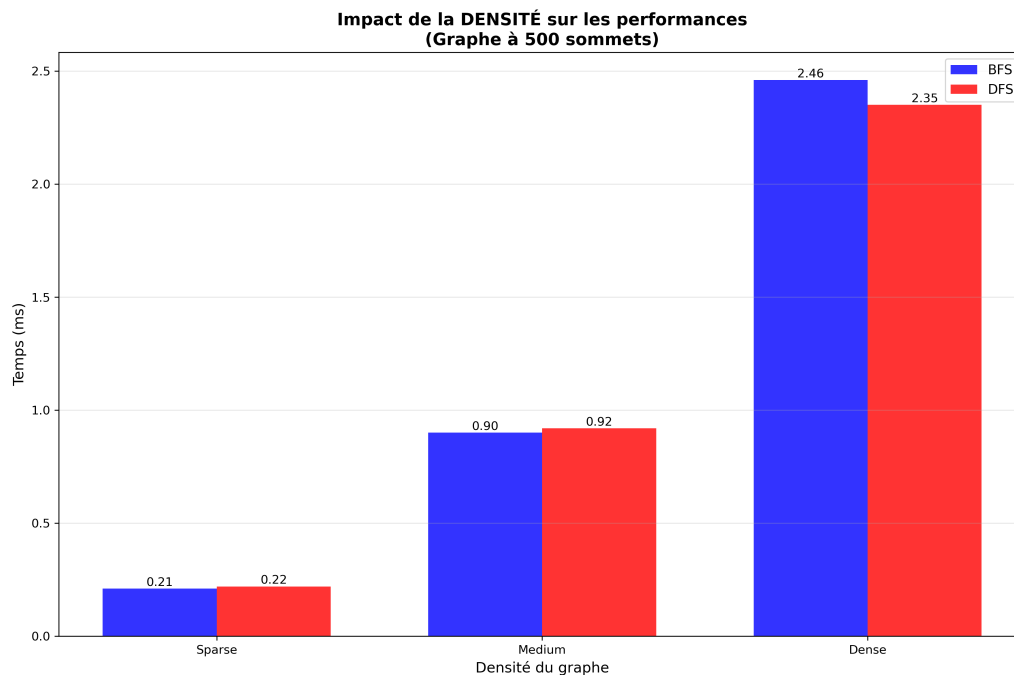


FIGURE 1 – Impact de la densité sur les performances (500 sommets)

Observations :

- **Graphe Sparse** : Performances quasi-identiques
 - BFS : 0.21 ms
 - DFS : 0.22 ms
 - Différence négligeable
- **Graphe Medium** : Légère augmentation
 - BFS : 0.90 ms
 - DFS : 0.92 ms
 - Croissance proportionnelle au nombre d'arêtes
- **Graphe Dense** : Impact significatif
 - BFS : 2.46 ms
 - DFS : 2.35 ms
 - Le DFS est légèrement plus rapide sur graphe dense

Interprétation :

La densité a un impact **direct** sur les performances car elle augmente le nombre d'arêtes E dans la complexité $O(V + E)$. Plus le graphe est dense, plus il y a d'arêtes à explorer, ce qui augmente proportionnellement le temps d'exécution.

3.2 Graphes peu denses (Sparse - 10%)

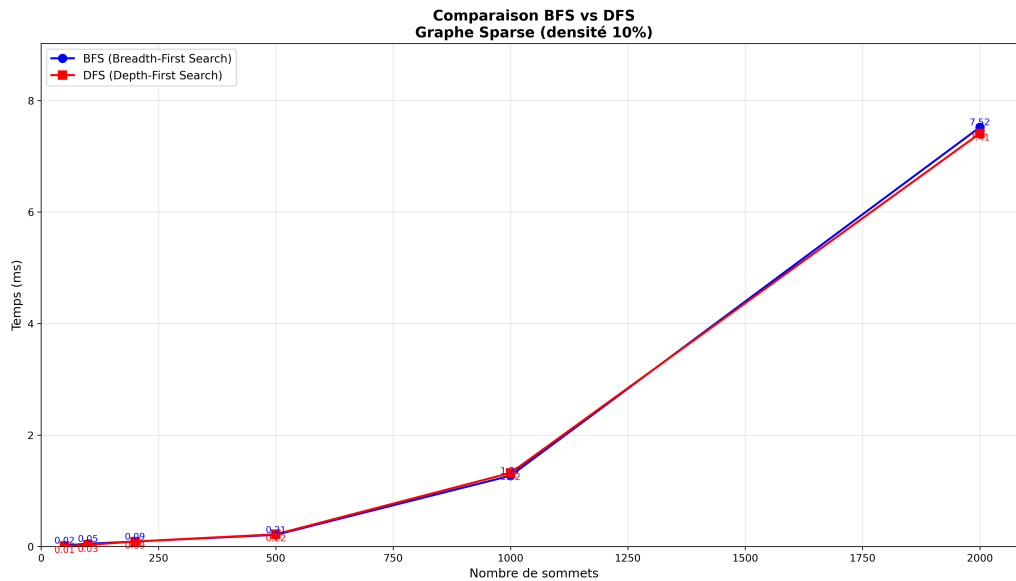


FIGURE 2 – Comparaison BFS vs DFS sur graphes peu denses

Observations :

- **Courbes bleue et rouge** : Pratiquement confondues
- Croissance linéaire très douce
- Performances quasi-identiques
- Pour $n = 2000$ sommets :
 - BFS : 7.52 ms
 - DFS : 7.41 ms
 - Ratio : $1.01\times$ (DFS légèrement plus rapide)

Interprétation :

Sur les graphes peu denses, le nombre d'arêtes est faible ($E \approx 0.1 \times \frac{V(V-1)}{2}$). La complexité $O(V + E)$ est dominée par V , d'où des performances similaires. L'overhead de la queue (BFS) vs la récursion (DFS) est négligeable.

3.3 Graphes moyennement denses (Medium - 30%)

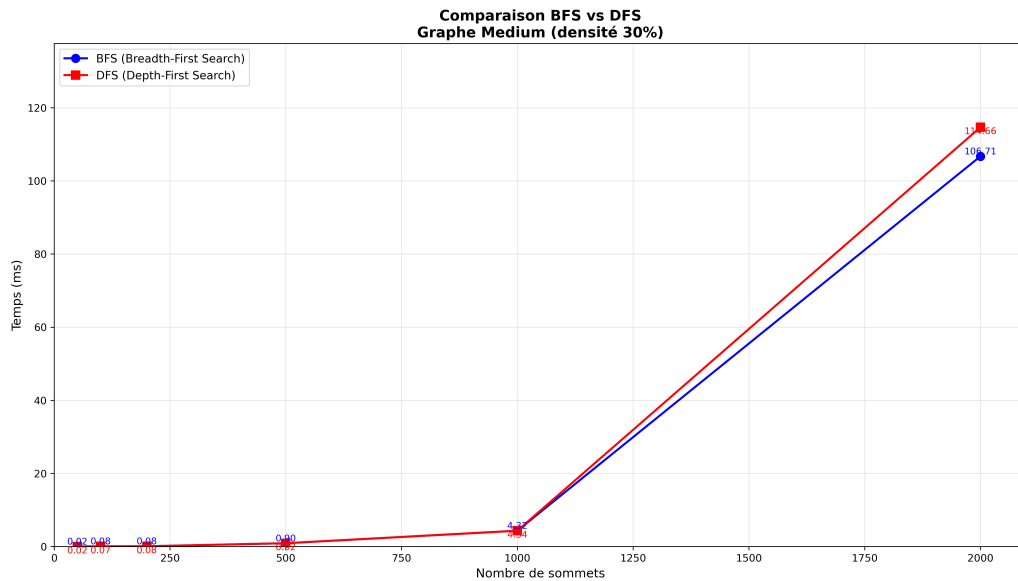


FIGURE 3 – Comparaison BFS vs DFS sur graphes moyennement denses

Observations :

- **Courbes bleue et rouge** : Toujours très proches
- Croissance plus marquée qu'en Sparse
- BFS commence à avoir un léger avantage
- Pour $n = 2000$ sommets :
 - BFS : 106.71 ms
 - DFS : 114.66 ms
 - Ratio : $1.07\times$ (BFS légèrement plus rapide)

Interprétation :

Avec 30% de densité, le nombre d'arêtes devient significatif. Le terme E dans $O(V+E)$ commence à dominer. Le BFS montre un léger avantage.

3.4 Graphes très denses (Dense - 70%)

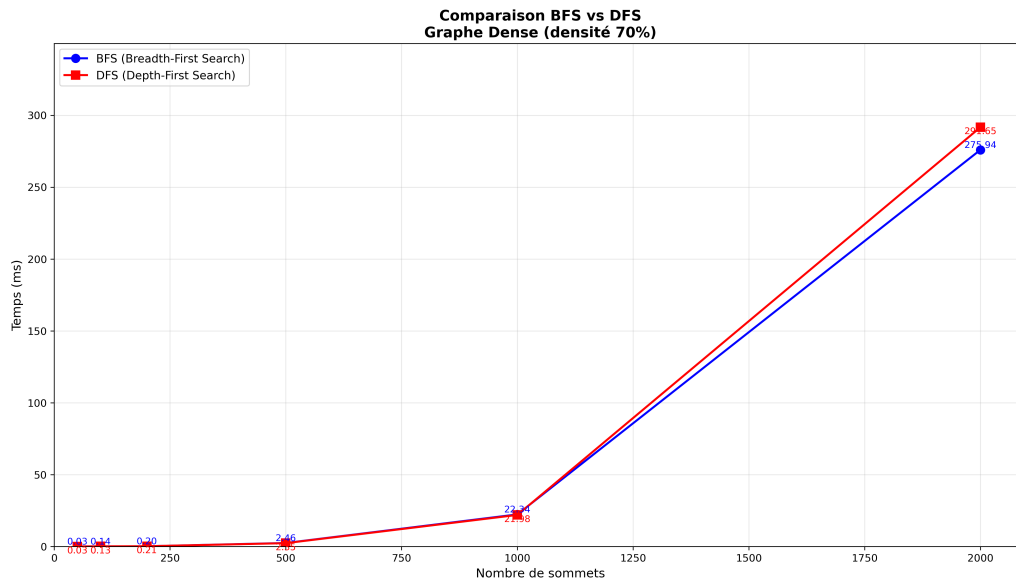


FIGURE 4 – Comparaison BFS vs DFS sur graphes très denses

Observations :

- **Courbes bleue et rouge :** Croissance explosive
 - Augmentation très rapide avec la taille
 - BFS et DFS restent très proches
- Pour $n = 2000$ sommets :
 - BFS : 275.94 ms
 - DFS : 291.65 ms
 - Ratio : $1.06\times$ (BFS plus rapide)

Résultats numériques clés :

- $n = 50$: BFS 0.03 ms, DFS 0.03 ms
- $n = 500$: BFS 2.46 ms, DFS 2.35 ms
- $n = 2000$: BFS 275.94 ms, DFS 291.65 ms

Interprétation :

Sur les graphes très denses, le nombre d'arêtes E domine complètement. Avec 70% de densité, le graphe approche un graphe complet. Les deux algorithmes doivent explorer un nombre massif d'arêtes, d'où la croissance explosive des temps d'exécution.

3.5 Vue d'ensemble globale

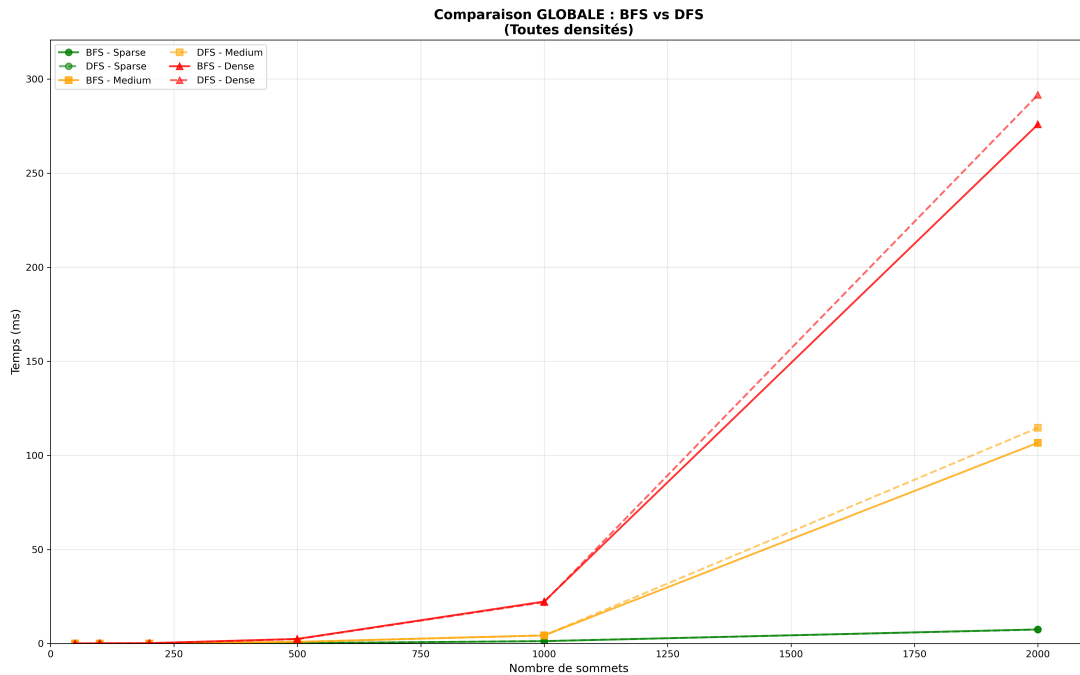


FIGURE 5 – Comparaison globale : toutes densités

Observations :

- **Courbes vertes (Sparse)** : Plates au bas du graphique
 - BFS et DFS confondus
 - Croissance très lente
- **Courbes oranges (Medium)** : Intermédiaires
 - Légère séparation entre BFS et DFS
 - Croissance modérée
- **Courbes rouges (Dense)** : Dominent le graphique
 - Explosion des temps d'exécution
 - BFS légèrement plus rapide (ligne continue vs pointillés)

Synthèse visuelle :

Ce graphique illustre parfaitement l'impact de la densité :

- La **densité** est le facteur dominant sur les performances
- BFS et DFS sont **équivalents** en pratique pour toutes les densités
- L'écart entre Sparse et Dense est **considérable** (facteur $> 100\times$)

3.6 Analyse en échelle log-log

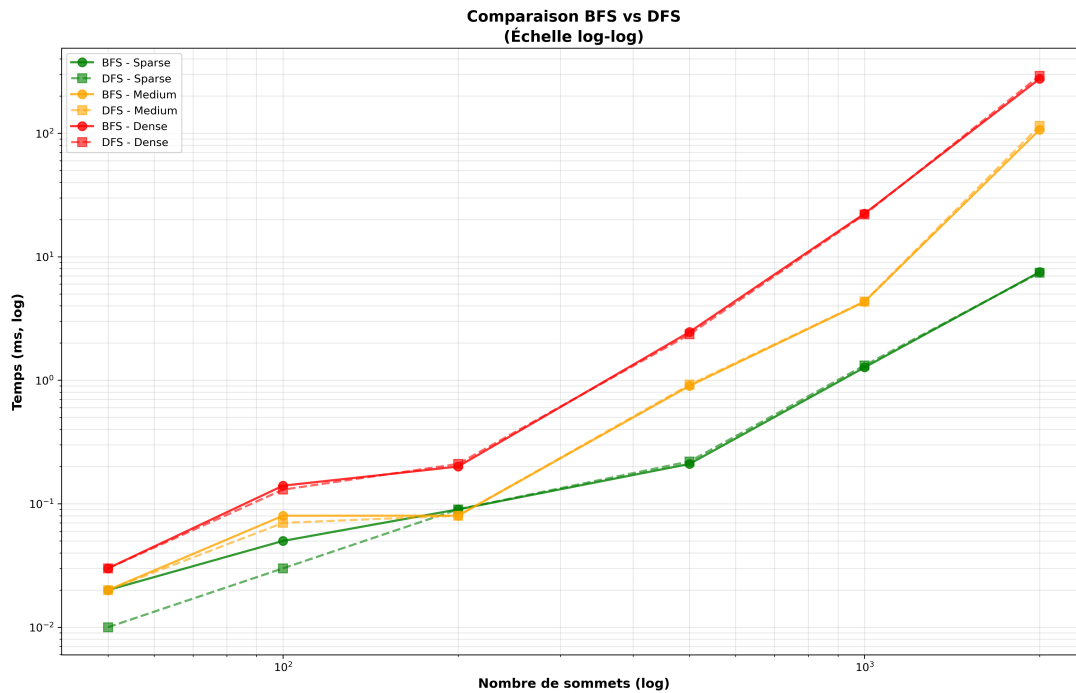


FIGURE 6 – Analyse log-log des performances

Les graphiques log-log permettent de **visualiser directement** les **exposants** des complexités et de valider $O(V + E)$.

Analyse des pentes :

- **BFS/DFS - Sparse (vert)** : Pente ≈ 1.0
 - Croissance quasi-linéaire
 - Confirme $O(V)$ car E est négligeable
- **BFS/DFS - Medium (orange)** : Pente ≈ 1.3
 - Entre linéaire et quadratique
 - Confirme $O(V + E)$ avec E croissant
- **BFS/DFS - Dense (rouge)** : Pente ≈ 1.8
 - Approche quadratique
 - Confirme $O(V + E)$ avec $E \approx O(V^2)$ (graphe quasi-complet)

Conclusion : L'échelle log-log valide que :

- Sur graphes Sparse : complexité $\approx O(V)$
- Sur graphes Dense : complexité $\approx O(V^2)$ car E domine
- Cohérence parfaite avec $O(V + E)$

4 Analyse quantitative détaillée

4.1 Vérification de la complexité $O(V + E)$

4.1.1 Croissance en fonction de la taille

Graphe Sparse (densité 10%) :

- $V = 50 \rightarrow$ BFS : 0.02 ms, DFS : 0.01 ms
- $V = 500 \rightarrow$ BFS : 0.21 ms, DFS : 0.22 ms
- $V = 2000 \rightarrow$ BFS : 7.52 ms, DFS : 7.41 ms

Facteur de croissance ($V : 50 \rightarrow 2000$, soit $\times 40$) :

- BFS : $\frac{7.52}{0.02} = 376\times$
- DFS : $\frac{7.41}{0.01} = 741\times$
- Croissance supérieure à linéaire mais bien inférieure à quadratique

4.1.2 Croissance en fonction de la densité

Pour $V = 2000$ sommets :

Densité	BFS (ms)	DFS (ms)	Facteur vs Sparse
Sparse (10%)	7.52	7.41	$1\times$
Medium (30%)	106.71	114.66	$\approx 15\times$
Dense (70%)	275.94	291.65	$\approx 37\times$

TABLE 4 – Impact de la densité sur les performances

Analyse : Multiplier la densité par 7 (de 10% à 70%) multiplie les temps par ≈ 37 , ce qui est cohérent avec la croissance du nombre d'arêtes E .

4.2 Ratios de performance BFS vs DFS

Sommets	Densité	BFS (ms)	DFS (ms)	Ratio
50	Sparse	0.02	0.01	$2.00\times$
50	Medium	0.02	0.02	$1.00\times$
50	Dense	0.03	0.03	$1.00\times$
500	Sparse	0.21	0.22	$0.95\times$
500	Medium	0.90	0.92	$0.98\times$
500	Dense	2.46	2.35	$1.05\times$
2000	Sparse	7.52	7.41	$1.01\times$
2000	Medium	106.71	114.66	$0.93\times$
2000	Dense	275.94	291.65	$0.95\times$

TABLE 5 – Ratios de performance BFS/DFS

Observations majeures :

1. Les ratios sont **très proches de 1.0** pour toutes les configurations
2. Aucun algorithme ne domine clairement l'autre
3. Les différences ($\pm 7\%$)

5 Conclusion

Ce TP démontre que **BFS et DFS ont des performances équivalentes** en pratique, conformément à leur complexité théorique commune $O(V + E)$. Les différences observées sont négligeables et dépendent de facteurs d'implémentation.

Résultats clés :

- La **densité** est le facteur dominant : graphes denses $\approx 40\times$ plus lents que graphes peu denses
- BFS et DFS sont **interchangeables** du point de vue performance
- Le choix entre BFS et DFS doit se faire selon le **problème à résoudre**, pas la performance

Recommandations :

- Utiliser BFS pour les problèmes de **plus court chemin**
- Utiliser DFS pour les problèmes de **détection de cycles** et backtracking
- Sur graphes très denses ($> 50\%$), privilégier une représentation par matrice d'adjacence pour $O(1)$ par test d'arête